EA Repository Workbench: User Guide (detailed)

Nov 2, 2026 · @Gaurav

1. About this guide

This guide explains every part of the EA Repository Workbench 1.1 for the architects who use it day to day, with step-by-step procedures and worked examples from the sample model. Installing and creating models is covered in the Setup Guide.

How it is organised. Sections 2 and 3 explain the ideas and the screens. Sections 4 to 6 cover everyday work: entities, relationships and validation. Sections 7 to 13 cover one capability each: the metamodel, sync, history, import and export, reports, diagrams, and parent and child models. Section 14 is reference material; section 15 collects good practice and frequent questions.

Conventions.

- **Bold** names a button, menu item or sheet you click or open, for example **Validate All** or **SYNC_Review**.
- *Italic* names a field on a form or a column on a sheet.
- A path such as Diagrams\Export\ is relative to the model folder.
- "Home" means the Home sheet of the workbook you are in. Buttons that exist on both Home sheets are marked *(both Homes)*.
- Examples use the sample model shipped in EA_Model_Sample: an insurer with 14 entities such as Claims Platform (APP-0002), Claims Management (CAP-0001) and Claims Department (BAC-0001).

What you need. Windows desktop Excel with macros enabled for the model folder. draw.io Desktop for diagrams and Power BI Desktop for Power BI reports are optional.

2. Concepts

The Workbench stores an architecture as entities joined by relationships, and a metamodel that says which entities, attributes and relationships are allowed. Everything else (sheets, forms, diagrams, reports) is a view of that store.

Term	Meaning	In the sample
Model	One architecture repository; one folder on disk	EA_Model_Sample
Metamodel	The rules of the model: entity types, attribute types, which attributes each type has, relationship types, which relationships are allowed, validation rules, lists	ArchiMate 4: 42 element types, 11 relationship types
Entity type	A kind of thing, with a domain (Business, Application…), an aspect and a Display ID prefix	Application Component, prefix APP
Entity	One thing in the architecture	Claims Platform
Attribute	A property of an entity. Four system attributes (Name, Description, Status, Repository Area) are on every entity; the rest apply per type through mappings	Owner, Lifecycle, Vendor
Mapping	"This attribute applies to this type", with whether it is required, its order and its section	Owner is required for Application Component
Relationship	A typed, directed link from a source entity to a target entity	Claims Platform *serves* Claims Department
Allowed relationship	A rule in the metamodel that a source type may have a relationship of a type to a target type	Application Component may *realize* Capability
Rule	A validation check on an attribute, as an error or a warning	Criticality must be 1 to 5

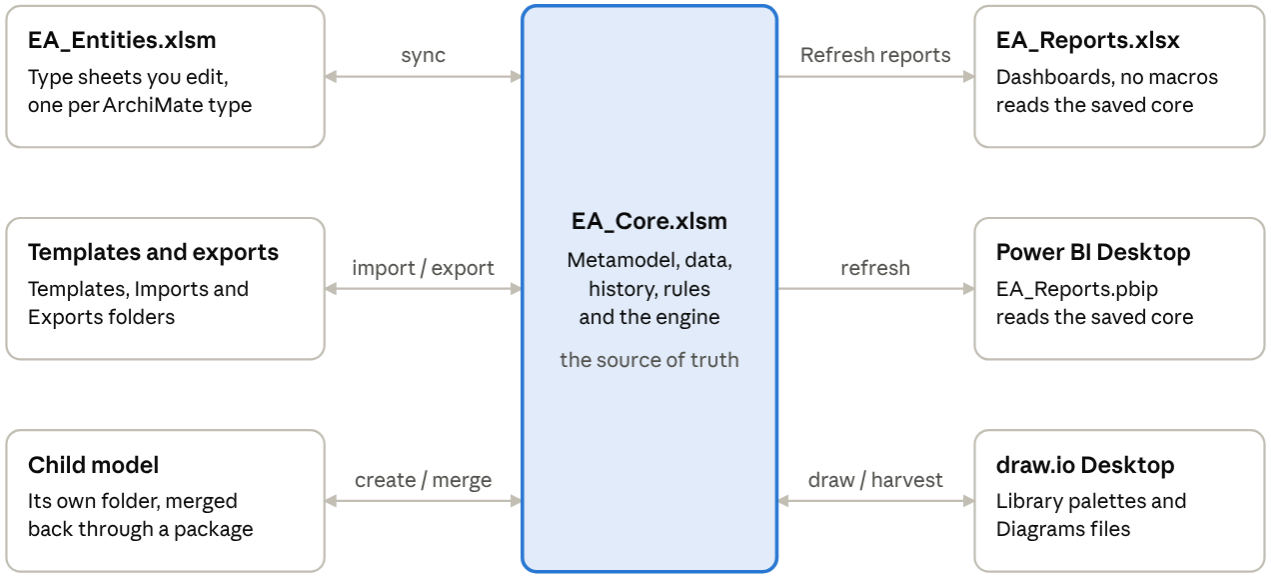
Two identifiers. Every entity has an *Entity ID*, a GUID that never changes and is used for every link, import, diagram and merge. It also has a *Display ID* such as APP-0002, which people read and type. Display IDs come from the type's prefix and a counter, and are never reused.

Status instead of delete. Entities and relationships are never deleted; set *Status* to **Retired** instead. Retired entities stay in history, compare and restore, are left out of new diagrams and libraries, and are flagged on diagrams that still show them.

Repository areas. *Repository Area* records where an entity belongs in the TOGAF architecture repository: Architecture Metamodel, Architecture Capability, Architecture Landscape (the default), Standards Information Base, Reference Library, Governance Log, Architecture Requirements Repository, or Solutions Landscape.

Two workbooks, one truth. EA_Core holds the metamodel, the master data (DATA_Master, one wide row per entity; DATA_Relationships, one row per relationship) and all history; it is the source of truth. EA_Entities is the workbook you edit: one type sheet per entity type, painted from EA_Core and kept in step with it by sync (section 8). Reports, templates, diagrams and child models all read or write EA_Core through files in the model folder.

EA_Core holds the model; the other tools reach it through files



the Workbench at a glance · one core, six connections

EA_Reports and Power BI read the last saved EA_Core, so save it (or use Refresh reports) before refreshing them.

3. Getting around

You open EA_Entities, work on its type sheets, and reach every other feature from the buttons on the two Home sheets.

Opening and closing

1. Open **EA_Entities.xlsm** from the model folder. If Excel shows a yellow bar, click **Enable Content**.
2. EA_Entities opens EA_Core in the background and runs a quick sync (a few seconds). A banner at the top of Home shows anything that needs attention, such as differences waiting on SYNC_Review.
3. Work, and save EA_Entities as often as you like; each save sends your edits to EA_Core.
4. Close EA_Entities. EA_Core closes too if EA_Entities opened it. If edits have not reached EA_Core yet, you are asked to save first.

Only one model can be open at a time: every model has files called EA_Core.xlsm and EA_Entities.xlsm, and Excel cannot open two files with the same name. Opening a second model shows a message asking you to close the first.

EA_Entities Home

The left of Home shows the model name and entity counts; the middle lists every type sheet, with its domain, as a link. The buttons:

Button	Does	Section
Add entity (form)	Opens the entity form for a new entity	4
Add relationship	Opens the relationship form	5
Save edits to EA_Core	Sends queued edits now, without saving the file	8
Sync Now	Full sync of both workbooks, then Validate All	8
Version history	Snapshots, compare, restore, entity history	9
Entity detail	Detail of the selected entities	11
Impact	Everything within n relationships of one entity	11
Import / export	Templates, import and export	10
Refresh reports	Builds or refreshes EA_Reports.xlsx	11
Diagrams (draw.io)	Generate, harvest, update and export diagrams	12
Parent and child models	Create a child, prepare a package, merge	13
Validate All	Checks the whole repository	6
Refresh type sheets	Repaints every type sheet from EA_Core	8

Button	Does	Section
Open EA_Core	Brings EA_Core to the front	—

EA_Core Home

EA_Core's Home has the same features plus the metamodel tools: **Add / edit type**, **Define rule**, **Apply metamodel changes** (section 7), **Review differences** (SYNC_Review, section 8), **Push EA_Entities edits** and **Open EA_Entities**. Most people only open EA_Core for metamodel work, SYNC_Review and SYS_Issues.

A type sheet

Each type sheet, such as **T_ApplicationComponent**, holds one row per entity of that type. The tab colour shows the domain.

Column	Holds	Editable
Display ID	APP-0002; assigned when a new row is saved	No
Name	The entity name (bold)	Yes
Description	Free text	Yes
Status	Active or Retired (dropdown)	Yes
Repository Area	Dropdown of the eight areas	Yes
One column per mapped attribute	Header ends with * when required; hover over a header for its description	Yes

Row 1 (column keys) and three technical columns (Entity ID, row kind, paint hash) are hidden; do not unhide or edit them. Rows and columns are frozen so the Name stays in view.

Under each entity sit its relationship rows, hidden until you click the **+** in the left margin. Each shows an arrow and verb (→ serves for outgoing, ← served by for incoming), the related entity as a link, its type, and the relationship label. Click the link to jump to the related entity.

Protection

Sheets are protected so formulas, keys and system columns cannot be broken by accident; the cells you are meant to edit are unlocked. You never need the password for normal work. If you must unprotect a sheet (Review > Unprotect Sheet), the password is

in SYS_Settings under *ProtectPassword*; the Workbench protects it again the next time it runs.

4. Working with entities

You can add and change entities directly on a type sheet, which is fastest for many rows, or through the entity form, which shows every attribute with its rules and sections.

On a type sheet

Add an entity.

1. Open the type sheet, for example **T_ApplicationComponent** (click its name on Home).
2. Go to the first blank row below the data and type the *Name*.
3. Fill the other columns. Required columns end with *; dropdown columns offer only valid values.
4. Save EA_Entities (Ctrl+S). The row is validated, written to EA_Core with history, and gets its Display ID, for example APP-0004.

Change an entity. Type over any editable cell and save. Only the cells you changed are sent; each change is a line in the entity's history.

What you see while typing. Each cell is checked as you leave it. A red cell is an error, an amber cell a warning; hover over it to read the message (section 6). A row with an error is not sent to EA_Core: it stays highlighted and queued, and is sent at the next save after you fix it. Warnings never stop a save.

Pasting. You can paste blocks of values. Each pasted cell is checked like a typed one. Paste values only (Paste Special > Values) so the sheet's formats and dropdowns stay in place.

With the entity form

Open it with **Add entity (form)** on Home, or right-click an entity row > **Edit this entity**. The form is the sheet FORM_Entity in EA_Core.

Field	What to enter
Entity type *	The type; choosing it redraws the form with that type's attributes
Existing entity (blank = new)	Pick an entity to edit it, or leave blank to add one
Display ID	Shown only; "(assigned on save)" for a new entity

Field	What to enter
Name *	Up to 120 characters; a duplicate name within the type gives a warning
Description	Free text
Status	Active or Retired
Repository area	One of the eight areas; Architecture Landscape by default
Attribute fields	One per mapped attribute, grouped under section headings, required ones marked *; dates, numbers and lists are checked as on the sheet. An entity reference field takes a Display ID, Entity ID or exact name

Buttons on the right: **Save entity** checks every field and saves (messages appear beside the fields); **New <type>** clears the form for another entity of the same type; **Retire entity** sets the status to Retired after you confirm; **Add relationship** opens the relationship form with this entity as the source; **Back to Home**.

Retiring and restoring

To take an entity out of use, set *Status* to **Retired** on the sheet or click **Retire entity** on the form. The row stays on its type sheet (so you can see and reverse it), its relationships stay, reports count it as retired, and new diagrams and the Model Library leave it out. To bring it back, set *Status* to Active.

Deleting a row on a type sheet does not delete the entity: the next sync puts the row back. The only way to remove entities from a model is to leave them out of a child model (section 13).

Worked example: add an application

The claims team has a new fraud-scoring system to record.

1. Open **T_ApplicationComponent**. Below Policy Administration System, Claims Platform and Customer Portal, type *Name* **Fraud Engine**.
2. Type *Description* "Scores claims for fraud risk". *Status* and *Repository Area* fill with Active and Architecture Landscape when the row is saved.
3. Leave *Owner* blank and pick *Lifecycle* **Plan**. Save: the Owner cell turns red with "Owner is required", and the row is not sent.
4. Type *Owner* **Claims IT**, *Criticality* **4**, *Vendor* **In-house**, *Version* **1.0**, *Hosting* **PaaS**. Save.

5. The row gets Display ID **APP-0004**. Right-click it > **History of this entity** to see one line per value just entered, with you as the user and the source "EA_Entities" (changes made on the form show "Form").

5. Working with relationships

A relationship links a source entity to a target entity with one of the 11 ArchiMate relationship types; the metamodel decides which combinations are allowed, and the form only offers those.

The relationship types

Relationship	Read as	Use for
Composition	A is composed of B	Parts that cannot exist without the whole
Aggregation	A aggregates B	Parts that can exist on their own (Grouping, Plateau, Location)
Assignment	A is assigned to B	Who performs what: an actor or component to a process or function
Realization	A realizes B	A process realizes a service; an application realizes a capability
Serving	A serves B	A service or application serves a user or another element
Access	A accesses B	Behaviour or active structure reads or writes data
Influence	A influences B	Between motivation elements
Association	A is associated with B	An unspecified link; the only undirected type
Triggering	A triggers B	Order between processes or events
Flow	A flows to B	Transfer between behaviour elements
Specialization	A specializes B	A is a kind of B (same type)

Add a relationship with the form

1. Right-click the source entity's row > **Add relationship from this entity**, or click **Add relationship** on Home.
2. On FORM_Relationship, fill the fields top down; each choice narrows the next:

- *Source type* * and *Source entity* * (filled in when you came from a row).
- *Relationship* *: only relationship types allowed from that source type.
- *Target type* *: only types allowed for that relationship from that source.
- *Target entity* *: the entities of that type.
- *Label* and *Description* (optional): shown on the type sheets and diagrams.

3. Click **Save relationship**. A duplicate (same type, source and target) is refused.

The relationship then appears under both entities: as → *verb* under the source and ← *inverse verb* under the target (for example → serves and ← served by).

See and follow relationships

Click **+** in the left margin of a row to open its relationship rows; click a related name to jump to it. **Entity detail** and **Impact** (section 11) show relationships across many hops.

Change or retire a relationship

Click **+** beside the entity, right-click the relationship row, and choose **Change or retire this relationship**. A message shows the relationship: **Yes** changes its label, **No** retires it, **Cancel** leaves it. A retired relationship disappears from the + rows, diagrams and reports but stays in history. To connect different entities, retire the old relationship and add a new one; relationship rows themselves are read-only.

Allowed relationships

The rules live in MM_EntityRelationships and are edited on the relationship type form (section 7). A rule names a source, a relationship type and a target, and each side can be a type code, `*` (any type), `SAME` (the same type as the source) or `G:<group>` (any type in a group in MM_TypeGroups, such as G:INT_ACTIVE for internal active structure). Validate All reports any existing relationship that no rule allows.

The 117 rules shipped are derived from the ArchiMate 4 generic metamodel and marked *ArchiMate4-derived (verify)*. Check them against the relationship tables in your copy of the ArchiMate 4 specification.

Worked example: connect the new application

1. Right-click **Fraud Engine** on T_ApplicationComponent > **Add relationship from this entity**.
2. Pick *Relationship* **Serving**, *Target type* **Business Actor**, *Target entity* **Claims Department**, *Label* "fraud scores". Save.
3. Add a second one: **Realization** to **Capability** *Claims Management*.

4. Try *Relationship* **Access** with *Target type* **Business Actor**: Business Actor is not offered, because applications access data objects, not actors.
5. Click + beside Fraud Engine: two rows, → serves Claims Department and → realizes Claims Management. On T_BusinessActor, Claims Department now shows ← served by Fraud Engine.

6. Validation

Every value is checked when you type it, when it is imported or harvested, and again by Validate All; errors stop a row from being saved, warnings only flag it.

Three layers of checks

1. **Applicability.** A value in an attribute that is not mapped to the entity's type is an error. In DATA_Master those cells are shaded grey.
2. **Data type.** The value must fit the attribute's type (table below).
3. **Rules.** The active rules in MM_Rules for that attribute, either for every type or only for one type.

Data type	Accepts	Example
Text, LongText	Any text	Head of Claims IT
Integer	Whole numbers	4
Decimal, Currency, Percent	Numbers	3100000
Date	A date (shown yyyy-mm-dd)	2020-07-15
Boolean	Y, N, TRUE or FALSE	Y
List	A value from the attribute's list in MM_Lists	Production
URL	A web address	https://wiki.example.com/claims
Email	An e-mail address	claims-it@example.com
EntityRef	The Display ID, Entity ID or exact name of another entity	APP-0002

Rule types

Rule type	Parameter 1	Parameter 2	Checks
Mandatory	—	—	A value is present
RequiredIf	Other attribute	Value	Required when the other attribute has that value
List	List name	—	The value is in the list
MinValue / MaxValue	Limit	—	Number at least / at most the limit
Range	Minimum	Maximum	Number within the range
MinLength / MaxLength	Characters	—	Text length
Regex	Regular expression	—	Text matches the pattern
DateRange	Earliest (yyyy-mm-dd)	Latest	Date within the range
DateOrder	Other date attribute	—	This date is after the other one
Unique	Type or Model	—	No other entity (of the type, or in the model) has the same value
EntityRef	Allowed type codes, comma separated	—	The value points at an existing entity of those types

Required attributes are set on the type's mapping (section 7), not as rules. Regex rules use the Windows VBScript regular-expression engine; where it is missing, they are skipped and Validate All says so.

The rules shipped with the sample

Attribute	Rule	Severity	Message
Name	Mandatory	Error	Name is required
Name	MaxLength 120	Error	Name must be 120 characters or fewer

Attribute	Rule	Severity	Message
Name	Unique within type	Warning	Another entity of this type has the same name
Status	List Status	Error	Status must be Active or Retired
Criticality	Range 1–5	Error	Criticality must be a whole number from 1 to 5
Version	Regex like 4.2.1	Warning	Version should look like 4.2 or 4.2.1
Retirement Date	DateOrder after Go-live Date	Error	Retirement date must be after the go-live date
Retirement Date	RequiredIf Lifecycle = Retiring	Warning	Set a retirement date for a retiring element
End Date	DateOrder after Start Date	Error	End date must be after the start date
Annual Cost	MinValue 0	Error	Annual cost cannot be negative
Process Level	Range 0–5	Error	Process level must be from 0 to 5
Capability Level	Range 1–4	Error	Capability level must be from 1 to 4
URL	Regex http(s)://	Warning	URL should start with http:// or https://
Go-live Date	DateRange 1990–2100	Warning	Go-live date looks out of range

Validate All and SYS_Issues

Validate All *(both Homes)* re-checks every entity and relationship, including relationships that no allowed-relationship rule covers. It writes every issue to **SYS_Issues** in EA_Core with the columns *Found*, *Sheet*, *RecordID*, *DisplayID*, *Name*, *TypeCode*, *AttrCode*, *RuleType*, *Severity*, *Message* and *Go to* (a link to the cell), and colours the cells on the type sheets. Sync Now and Refresh reports run it too, and the Data quality page of EA_Reports lists the same issues.

Run it after metamodel changes and before a review or a merge. Fix errors first; warnings are judgement calls.

7. The metamodel

The metamodel is data in EA_Core's MM_ sheets, seeded with ArchiMate 4; you change it with two forms on EA_Core's Home, then apply it so the sheets, columns and checks follow. A snapshot is taken automatically before every metamodel change.

The metamodel sheets

Sheet	One row per	Edited with
MM_EntityTypes	Entity type: code, name, domain, aspect, sheet, ID prefix, style, status	Add / edit type > Entity type
MM_AttributeTypes	Attribute: code, name, data type, list, unit, default, description	Add / edit type > Attribute type
MM_EntityAttributes	Mapping of an attribute to a type: required, display order, section	The grids on both type forms
MM_RelationshipTypes	Relationship type: code, verb, category, directed	Add / edit type > Relationship type
MM_EntityRelationships	Allowed relationship: source, relationship, target	The grid on the relationship type form
MM_TypeGroups	Group membership used by G:<group> rules	Directly in the sheet
MM_Rules	Validation rule	Define rule
MM_Lists	Dropdown list values, in order	New list field on the attribute form; directly in the sheet
MM_Styles	Domain fill colour and draw.io style per type and relationship	Directly in the sheet

Names, descriptions, units and rule messages can be corrected directly in the MM_ sheets; everything else goes through the forms.

Add or edit a type: the Add / edit type form

Choose *Kind*: **Entity type**, **Attribute type** or **Relationship type**. Leave *Existing* blank for a new one, or pick one to edit it.

Entity type. *Type code* * (no spaces, fixed once saved, for example Contract), *Name* *, *Domain* * (Common, Business, Application, Technology, Strategy, Motivation, Implementation & Migration), *Aspect*, *Display ID prefix* * (three letters such as CON, fixed once saved), *Description*, *Status*. For a new type, *Copy attribute mappings from* fills the grid from a similar type. The grid **Attributes for this type** lists every attribute with *Map? (Y/N)*, *Mandatory? (Y/N)* and *Section* (the heading it appears under on the entity form).

Attribute type. *Attribute code* * (fixed once saved), *Name* *, *Data type* * (fixed once saved), *List name* for a List attribute, or *New list: name and values* in the form `ListName: value 1; value 2; value 3`, *Unit*, *Default value*, *Description*, *Status*. The grid **Entity types that carry this attribute** maps it to types in one go.

Relationship type. *Relationship code* *, *Verb* * (shown on the sheets, such as "serves"), *Category* * (Structural, Dependency, Dynamic, Other), *Directed (Y/N)*, *Description*, *Status*. The grid **Allowed source and target types** lists the allowed combinations with *Source type*, *Target type*, *Min*, *Max* and *Status*; each side takes a type code, `*`, `SAME` or `G:<group>`.

Click **Save** on the form. Each change is logged on H_Metamodel.

Define a rule

On **Define rule**: pick the *Attribute* *, leave *Existing rule* blank for a new rule, then set *Only for type* (blank = every type), *Rule type* * (the hint beside *Parameter 1* explains the parameters), *Parameter 1*, *Parameter 2*, *Severity* * (Error or Warning), *Message* * (what the user sees) and *Status*. Type a sample in *Test value* and click **Test rule** to see whether it passes before you **Save rule**.

Apply metamodel changes

After changing types, attributes or mappings, click **Apply metamodel changes** on EA_Core's Home and confirm. It raises the schema version, adds any new DATA_Master columns, rebuilds dropdowns and checks, regenerates every type sheet in EA_Entities, and keeps all data. Then:

- run **Validate All**: existing entities may now miss a newly required attribute;
- rebuild the draw.io stencil (Diagrams (draw.io) > **Rebuild ArchiMate 4 stencil**), or let the next Sync Now do it;
- regenerate import templates you sent out before the change.

Retire types and attributes rather than deleting them, so history and old snapshots stay readable.

Worked example: add a Contract type

1. EA_Core Home > **Add / edit type**, *Kind* **Entity type**.
2. *Type code* **Contract**, *Name* **Contract**, *Domain* **Business**, *Aspect* **Passive Structure**, *Prefix* **CON**.
3. *Copy attribute mappings from* **Business Object**, then in the grid map **Owner** (Mandatory Y), **Start Date** and **End Date**. Save.
4. *Kind* **Relationship type**, *Existing* **Access**: in the allowed grid add *Source type* **ApplicationComponent**, *Target type* **Contract**, so applications can be recorded as reading or writing contracts. Save. (Association needs no rule: it is allowed between any two types.)
5. **Apply metamodel changes**. EA_Entities gets a T_Contract sheet; the entity form and templates offer Contract; End Date must come after Start Date because that rule applies to every type.

8. Sync

EA_Core is the source of truth and EA_Entities is kept in step with it automatically; you only decide something when the two disagree in a way the Workbench cannot settle on its own.

What happens when

When	What runs	What it does
You open EA_Entities or EA_Core	Quick sync	Sends queued edits; repaints rows that changed in EA_Core; fixes sheet structure and row order
You save EA_Entities	Push	Validates your edited rows and writes them to EA_Core with history; rows with errors stay queued
Save edits to EA_Core	Push	The same, without saving the file
Sync Now *(both Homes)*	Full sync	Compares every row in both workbooks, runs Validate All, archives large history, rebuilds the draw.io libraries and checks the views register

When	What runs	What it does
Refresh type sheets	Repaint	Repaints every type sheet from EA_Core; sheets with unsaved edits or open review items are skipped
A workbook was saved with macros off	Detected on the next open	The rows of that workbook are compared in full, as in Sync Now

Each row carries a hash of its values, so sync can tell which side changed a row since it was last in step: the sheet, EA_Core, both or neither.

SYNC_Review

Differences that need a decision are listed on **SYNC_Review** in EA_Core (Home > **Review differences**), one row per entity and attribute, showing the EA_Core value and the EA_Entities value side by side with the situation.

Situation	Default *Keep*	Why
Edited in EA_Entities outside the Workbench (macros off)	EA_Entities	Your edit was never sent; apply it
Changed in both EA_Core and EA_Entities	EA_Core	EA_Core is the source of truth
New row in EA_Entities	EA_Entities	Add the new entity
Row in EA_Entities not found in EA_Core	EA_Core	Remove the stray row from the sheet

For each row set *Keep* to **EA_Core**, **EA_Entities** or **Skip** (decide later), or click **Set all to EA_Core**. Then click **Apply decisions**: EA_Entities values are validated and written with history, EA_Core values are painted back onto the sheet. A type sheet with open items is not repainted until they are settled, so nothing on it is overwritten.

Working without the Workbench

You can open EA_Entities with macros off (on a Mac, Excel on the web, or by mistake) and edit it. Nothing is checked or sent at that point. The next time it is opened with macros on, the Workbench notices the outside save and lists the edits on SYNC_Review with *Keep* = EA_Entities, so they are applied after you review them.

Editing EA_Core's DATA_ sheets by hand is not supported: they are protected, and changes there bypass validation and history.

9. History and versions

The Workbench keeps two kinds of record: a line of change history for every value ever changed, and snapshots of the whole model that you can compare and restore.

Change history

Every create, update, retire, restore, import, harvest and merge writes one line per changed value to an H_ sheet in EA_Core: one sheet per entity type, plus **H_Relationships** and **H_Metamodel**. Each line records the timestamp (UTC), user, action, record ID, attribute, old value, new value, *Source* (EA_Entities, Form, Import <file>, Diagram <file>, Merge <child>, Restore) and a *Batch* ID that groups the changes made by one operation. History is never edited or deleted.

To read it for one entity, right-click its row > **History of this entity**, or **Version history** > **Entity history** and enter the Display ID. The result lists every change to the entity and its relationships, newest first, with the columns *Timestamp (UTC)*, *User*, *Action*, *Field*, *Old*, *New*, *Source* and *Batch*.

Archiving. When an H_ sheet passes 100,000 rows, Sync Now moves the oldest rows to History\Archive_<year>.xlsx, so the workbook stays fast. Archived history is still in the folder, but not in Entity history.

Snapshots

A snapshot is a values-only copy of the model saved in History: one file with EA_Core's metamodel, data and history, one with the type sheets of EA_Entities that hold entities. Snapshots are listed on **Version history** *(both Homes)* with *Version*, *Label*, *Taken (UTC)*, *By*, *Auto*, *Reason* and the entity and relationship counts.

Take one yourself with **Take snapshot** and a label, for example before a review board. The Workbench takes one automatically before every metamodel change, restore, import, diagram harvest and merge.

Compare

Select a snapshot in the list and click **Compare selected with current**, or click **Compare two versions** and pick two. **VERSION_Compare** lists every difference, with *Area* (entity, relationship or metamodel), *Change* (added, removed, changed), *ID*, *Display ID*, *Name*, *Type / Rel*, *Field*, *Old* and *New*.

Restore

- **Restore entity or type:** select a snapshot, enter a Display ID (one entity) or a type code (every entity of that type). The values come back from the snapshot; everything else is untouched.
- **Restore whole model:** replaces the metamodel, entities and relationships with the snapshot's, then regenerates the type sheets.

Both take a snapshot first, so a restore can itself be undone, and log every restored value in history with the source "Restore".

Worked example: undo a bad change

Someone overwrote Claims Platform's *Vendor* with "Duck Creek" by mistake last Thursday.

1. Right-click Claims Platform > **History of this entity**: the line shows Vendor, Guidewire → Duck Creek, who made it and when.
2. **Version history**: select the snapshot taken before that date and click **Compare selected with current** to confirm it is the only unwanted change on APP-0002.
3. Select the same snapshot, click **Restore entity or type**, enter **APP-0002**, confirm.
4. Vendor is Guidewire again; the restore is a new line in the history, and the mistake stays visible.

10. Templates, import and export

Templates let people without the Workbench supply data in a controlled format; import reads them back with a preview; export hands data out in Excel or CSV. All three are on **Templates, import, export** (EA_Core Home) or **Import / export** (EA_Entities Home).

Generate a template

On the form, section 1:

Field	Meaning
Entity type *	The type the template is for; one type per template
Pre-fill existing entities (Y/N)	Y lists every existing entity of the type with its values, for bulk updates; N gives an empty sheet for new entities
Include relationships sheet (Y/N)	Y adds sheets for relationships from this type

Click **Generate template**. The file is saved in Templates\ as `<Type>-template-<date>.xlsx` (with -prefilled for Y). It has no macros, so it can be e-mailed to anyone with Excel.

What is in a template.

Sheet	Contents
README	How to fill it in
Entities	One row per entity: Display ID, Name and every attribute of the type, with dropdowns for lists and Status; required columns marked *. A hidden column holds the Entity ID of pre-filled rows
Relationships	*Source name* (a dropdown of the names on Entities), *Relationship*, *Target type*, *Target name*, *Label*, *Description*; Relationship and Target type offer only allowed values
Allowed	The allowed relationship and target-type pairs for this type
Lists (hidden) and _meta (very hidden)	Dropdown values, and the model ID, type and schema version that import checks

Do not rename sheets or columns, and do not delete the hidden ones.

Import

1. Put the filled templates in Imports\ (created with the first template).
2. Click **Choose files and preview** and select one or more files. Nothing is written yet.
3. Read **IMPORT_Preview**: one row per changed field, with *File*, *Sheet*, *Row*, *Kind* (Entity, Relationship, File), *Action*, *EntityID*, *DisplayID*, *Name*, *TypeCode*, *Field*, *Current*, *Imported*, *Severity* and *Message*.
4. Click **Commit import**. A snapshot is taken, valid rows are written with history (source "Import <file>"), and the files move to Imports\Done\ with a date prefix. The preview list is cleared so the same import cannot run twice.

How rows are matched. By Entity ID first (pre-filled rows); otherwise by exact name within the type, ignoring case. A name used by two entities of the type is an error: add the Entity ID or rename one.

Blank cells. In a pre-filled template a cell you clear clears the value. In an empty template, blanks are ignored for entities that already exist, so a short list of names and one column can update just that column.

Action on the preview	Meaning
New / Added	A new entity or relationship (Added after commit)
Update / Updated	A changed field, with the current and imported value
Unchanged	Matched, nothing to change
Error	Skipped; *Message* says why, such as "Owner is required" or "... is not allowed"
Info	A warning, or an attribute that no longer applies to the type

File-level refusals. A file that is open in Excel, is not a Workbench template, belongs to another model, or is for a type that no longer exists is refused as a whole and left where it is.

Export

On the form, section 3: *Scope* * (All entities, One entity type, Around one entity, Full model with metamodel), *Entity type* (for one type), *Start entity Display ID* and *Relationship hops (1-5)* (for around one entity), *Include retired (Y/N)*, and *Format* * (Excel or CSV). Click **Export**.

An Excel export goes to Exports\ as one workbook: README, one sheet per entity type with display names as headers, Relationships (only those with both ends in the export), and, for the full model, every MM_ sheet. A CSV export is a folder in Exports\ with one UTF-8 file per sheet. Exports are values only and are not meant to be imported back; use templates for that.

Worked example: update owners in bulk

1. Generate a template for **Application Component** with *Pre-fill* **Y** and send it to the application portfolio manager.
2. They change Customer Portal's *Owner* to "Digital Platforms", clear Policy Administration System's *Version*, and add a row "Broker Portal" with *Lifecycle* Production but no Owner.
3. Save the returned file in Imports, click **Choose files and preview**.

4. IMPORT_Preview shows: Customer Portal Update Owner Digital Channels → Digital Platforms; Policy Administration System Update Version 10.2 → blank; Broker Portal Error "Owner is required".
5. Click **Commit import**: two updates are written, Broker Portal is skipped, the file moves to Imports\Done. Ask for the owner of Broker Portal and import it again.

11. Reports

Three kinds of report cover different needs: EA_Reports.xlsx for dashboards anyone can open, Entity detail and Impact for questions about particular entities, and Power BI for richer, shareable analysis.

EA_Reports.xlsx

Click **Refresh reports** *(both Homes)*. The Workbench runs Validate All, saves EA_Core, then creates EA_Reports.xlsx in the model folder the first time, or refreshes it, and opens it on the Dashboard.

EA_Reports has no macros. It reads the saved EA_Core in its own folder through Power Query, so anyone can open it on its own and use **Data > Refresh All**; nobody needs to open EA_Core. It shows the state of EA_Core when it was last saved. It needs Microsoft 365 or Excel 2021; if Excel asks about privacy levels, choose to ignore them for this file.

Page	Shows
Dashboard	Six figures across the top (Entities, Relationships, Errors, Warnings, Orphans, Data quality %) and charts of entities by domain and type, by lifecycle and by repository area
Entity list	Every entity with its type, domain and system fields; slicers filter by type and domain
Relationships	Every relationship with source and target names and types, verb and label
Matrix	A pivot of source entities against target entities; set the page filters *SourceType*, *TargetType* and *Relationship*, for example Application Component × Capability for Realization
Data quality	Every open issue from Validate All
Orphans	Entities with no relationships at all

Data quality % is the share of entities without an error. Do not type on the report sheets: a refresh rewrites them. Build your own analysis on new sheets, pointing at the tables tEntities, tRelationships, tIssues and tOrphans.

Entity detail

Select one or more entity rows on a type sheet and right-click > **Detail of selected entities**, or click **Entity detail** and type Display IDs separated by commas. **RPT_EntityDetail** in EA_Entities shows, for each entity, its type, Display ID, status, repository area and description; every attribute value; every relationship in both directions with links; and its recent changes from history. It is rebuilt each time; print it or copy it into a review pack.

Impact

Right-click an entity > **Impact of this entity**, or click **Impact** and type a Display ID, then give the number of hops (1 to 5). **RPT_Impact** lists every entity reachable within that many relationships, in either direction, grouped by hop, with how each was reached, for example "Claims Department, served by Claims Platform". Retired relationships are not followed. Use it for change impact ("what is affected if this application is replaced?") and for scoping a child model.

Power BI

Reports\EA_Reports.pbip is a Power BI project with the data model ready and the report pages left to you.

1. Open it in Power BI Desktop.
2. **Home > Transform data > Edit parameters**, set *ModelFolder* to the model folder, ending with a backslash.
3. **Home > Refresh.** Power BI reads the saved EA_Core.

Table	One row per
Entity	Entity, with the system fields and type code
EntityAttribute	Entity and filled attribute (long form: Attribute, Value)
EntityType	Entity type, with name, domain and aspect
Relationships	Relationship, linked to Entity (source) and Target Entity
Target Entity	Entity as a relationship target

Table	One row per
Issues	Validate All issue

Measures: Entities, Retired entities, Data quality %, Relationship count, Errors, Warnings. Because attributes are in long form, new attributes and types appear after a refresh with no change to the model. Reports\README_PowerBI.md describes five suggested pages (portfolio overview, application × capability, lifecycle and risk, data quality, drill-through), about 15 minutes to lay out. If the model folder moves, use **Rebuild Power BI project** in the Setup window, or change the parameter.

12. Diagrams with draw.io

Diagrams are drawn in draw.io Desktop and kept in step with the repository: every shape and connector carries the ID and type of the repository object it shows, so the Workbench can draw diagrams from the model and read new things back from them. Open the tools with **Diagrams (draw.io)** *(both Homes)*.

The two libraries (once per computer)

In draw.io: **File > Open Library from > Device**, pick Library\ArchiMate4.xml; repeat for Library\ModelLibrary.xml. Both stay in draw.io's left panel.

- **ArchiMate4:** 42 element shapes in domain colours, using draw.io's ArchiMate notation (square corners for structure, rounded for behaviour, cut corners for motivation), then 11 connectors, one per relationship type. Each shape is tagged with its type and, where the type has required attributes, carries them as blank properties.
- **ModelLibrary:** every active entity in the model as a ready-tagged shape, titled "Type: Name (Display ID)", ordered by type and name. It is rebuilt by every Sync Now and by **Rebuild Model Library**.

After a metamodel change, **Rebuild ArchiMate 4 stencil** (or Sync Now) and reopen the library in draw.io (right-click its title > Close, then open it again).

What is on a shape

Select a shape and press **Ctrl+M** (Edit Data) to see its properties: *ea_type* (type code), *ea_id* (Entity ID, blank on a new shape), *ea_display*, *ea_name* (the name when it was last in step) and *ea_model*. Connectors carry *ea_reltype* and *ea_rel_id*. Do not edit these by hand. You may add properties named after the type's attribute codes (Owner, Lifecycle, Vendor...); they are used when the shape creates a new entity.

Draw

- **A new element:** drag it from ArchiMate4, double-click and type its name. For a type with required attributes, press Ctrl+M and fill them in.
- **An existing element:** drag it from ModelLibrary. It is the real entity, so nothing new is created.
- **A relationship:** drag a connector from the bottom of ArchiMate4 and attach both ends to shapes (the ends turn green when attached). For Composition and Aggregation the diamond is the whole.
- **Notes, groups, colours and layout** are free: anything without an *ea_type* is ignored by the Workbench.

Generate a diagram from the repository

On the Diagrams form, section 1:

Field	Meaning
Diagram file name *	Saved as Diagrams<name>.drawio
Page name	A page in that file; a new page is added if it does not exist
Content *	Around one entity, One entity type, or All entities
Entity type	For one entity type
Start entity Display ID	For around one entity
Relationship hops (1-5)	How far around the start entity

Click **Generate / add to diagram**. Shapes are placed in bands by domain (Motivation, Strategy, Business, Common, Application, Technology, Implementation & Migration), six to a row, with every relationship between them drawn. Tidy the layout in draw.io.

Generating into a page that already exists keeps every shape where you put it: only entities and relationships missing from the page are added, below the existing content. Right-click entity rows on a type sheet > **Diagram of selected entities** does the same for a selection (one entity: it and its direct neighbours), on a new page.

Harvest a diagram into the repository

1. Save the diagram in draw.io and close it.
2. Click **Choose diagrams and preview** and pick one or more .drawio files.

3. Read **HARVEST_Review** (columns *File*, *Page*, *CellID*, *Kind*, *Action*, *EntityID*, *DisplayID*, *Name*, *TypeCode*, *Field*, *Current*, *Proposed*, *Severity*, *Message*).
4. Click **Commit harvest**: a snapshot is taken, the changes are written with history (source "Diagram <file>"), and the new IDs are written into the diagram file. Reopen it in draw.io.

On the diagram	Result
New shape from ArchiMate4 with a name	New entity; attribute properties from Ctrl+M are used; missing required values are an error with a hint
New shape named like an existing active entity of the same type	Linked to that entity; no duplicate
The same new name drawn twice	One entity, both shapes linked
Shape renamed on the diagram	Rename of the entity, checked like any name change
Shape whose entity was renamed in Excel	Information only; Update diagrams refreshes the label
New connector from ArchiMate4 between two repository shapes	New relationship, if allowed; an existing identical one is linked instead
Plain draw.io connector between repository shapes	Type taken from its line style if it matches one relationship; otherwise listed as ignored
Relationship ArchiMate does not allow	Error with the reason
Connector moved to other shapes after it was harvested	Warning; the relationship is not changed
Shape copied from another model's library	Error
Shape or connector deleted	Nothing; deleting in draw.io never deletes in the repository

If a diagram changed after the preview, the commit refuses and asks you to preview again. Diagrams must be saved uncompressed (the default in current draw.io); if a page is compressed, untick **File > Properties > Compressed** and save.

Keep diagrams current

- **Update diagrams from model** goes through every diagram in Diagrams: it refreshes names and Display IDs, and gives retired or deleted entities a red outline (removed again if they come back). Labels you changed on the diagram are kept for the next harvest. Close the diagrams in draw.io first.
- **Views register** (SYS_Views) lists every page generated, updated or harvested, with *File*, *Page*, *Entities*, *Relationships*, *Generated*, *Harvested*, *Status* and *Note*. *Status* is OK; Stale (entities retired, deleted or changed since the page was last in step); "n relationship(s) not drawn"; Page missing; or File missing. **Open selected diagram** opens the clicked row in draw.io.

Open and export

Open diagram starts draw.io on a file (type its name, or pick it). **Export images** writes every page of every diagram to Diagrams\Export\ as PNG, SVG or PDF using draw.io's own exporter, for slides, documents and Power BI. draw.io is found automatically; set *DrawioPath* in SYS_Settings if it is installed elsewhere.

Worked example: a claims view

1. Diagrams form: *Diagram file name* **Claims**, *Content* **Around one entity**, *Start* **APP-0002**, *Hops* **1** > **Generate / add to diagram** > open it. Claims Platform appears with Claims Management, Claim Record, Claims Department, Customer Portal and Digital first notification of loss, and their connectors.
2. In draw.io, drag **Application Component** from ArchiMate4, name it **Fraud Engine**, Ctrl+M: *Owner* Claims IT, *Lifecycle* Plan. Drag a **Serving** connector from it to Claims Department. Save and close.
3. **Choose diagrams and preview** > Claims.drawio: Fraud Engine New, the connector New. **Commit harvest**.
4. T_ApplicationComponent shows Fraud Engine with → serves Claims Department; Claims.drawio now carries its ID; the next Sync Now adds it to the ModelLibrary.

13. Parent and child models

A child model is a copy of part of a model for a project, a team or a consultant, worked on separately and merged back with a field-by-field review. Everything is on **Parent and child models** *(both Homes)*.

The life of a child

Step	Where	You do	The Workbench does
1. Create	Parent	Name the child, pick the scope, **Create child**	Copies the folder next to the parent; records the child on SYS_Children
2. First open	Child	Close the parent; open the child's EA_Entities.xlsm	Removes everything outside the scope, sets its own model ID and lineage, regenerates the type sheets, saves the merge base
3. Work	Child	Edit as in any model	Normal history, sync and validation
4. Package	Child	**Prepare merge package**, then close the child	Writes Merge\ChildPackage.xlsx
5. Preview	Parent	**Choose child and preview**, pick the package	Three-way comparison on MERGE_Review
6. Decide and apply	Parent	Set *Keep*, **Apply merge**	Snapshot, metamodel first, then entities, relationships, diagrams; moves the merge base forward
7. Repeat	Child	Carry on and package again	The next merge brings only what changed since

Parent and child are never open together: both have files called EA_Core.xlsm and EA_Entities.xlsm, and Excel cannot open two files with the same name. They meet only through the package and the base, which have their own names.

Scope

Scope	In the child	Fields
Around entities	The start entities and everything within the hops	*Start Display IDs, comma separated*; *Relationship hops (0-5)*
One entity type	Every active entity of the type	*Entity type*
Repository area	Every active entity in the area	*Repository area*

Scope	In the child	Fields
Whole model	Everything, including retired entities	—

Entities outside the scope that in-scope relationships point at are copied as **references**, so every link still resolves. Changes to references in the child are listed on the merge review but not merged. The child gets the full metamodel, the code, the libraries, the Power BI project, and every diagram whose shapes are all in the child.

The merge base

The base is History\MergeBase.xlsx in the child: a values-only copy of the child as it was created, or as it was at the last merge. Comparing base, parent and child tells the Workbench who changed what. After a merge where nothing new was skipped or failed, the package becomes the new base. Otherwise the base stays, so the skipped items come back next time; records already merged compare as equal and do not reappear.

How differences are decided

Base → parent → child	Row on MERGE_Review	Default *Keep*
Changed in the child only	Update	Child
Changed in the parent only	No row; the parent's value stays	—
Same change on both sides	No row	—
Different changes on both sides	Conflict	Parent
Entity or relationship new in the child	Add	Child
Retired in the parent, changed in the child	Conflict	Parent
Retired in the child, changed in the parent	Conflict on Status	Parent
Reference entity changed in the child	Info, not merged	Skip
Metamodel row new in the child	Add	Child
Metamodel row changed in the child only, or in both	Update, or Conflict	Child, or Parent
Diagram new in the child	Add (copied)	Child
Diagram changed in the child since the base	Copy, as <name>_child.drawio	Child

Decisions are per attribute: a parent edit to *Owner* and a child edit to *Lifecycle* on the same entity both survive. New entities keep their Entity ID, and their Display ID if it is free in the parent (otherwise they are renumbered). Diagram contents are never merged; a changed diagram arrives next to the parent's for you to compare.

MERGE_Review

Columns: *Kind* (Metamodel, Entity, Relationship, Diagram), *Action* (Add, Update, Conflict, Info, Copy), *RecordID*, *DisplayID*, *Name*, *TableOrType*, *Field*, *Base*, *Parent*, *Child*, *Keep* and *Note*. Set *Keep* on any row to **Parent** (leave the parent as it is), **Child** (take the child's value) or **Skip**. Then click **Apply merge**. If the package changed after the preview, the merge refuses and asks you to preview again. After applying, *Note* shows the result of each row, for example "Added as APP-0005" or "Not added: Owner is required". Every change is logged with the source "Merge <child name>".

Limits

- Merging runs from child to parent only. To pick up later parent changes, create a new child.
- A merge refuses a package from a model that is not this model's child.
- Child setup runs once, on the first open; if it fails it says so, and opening the child again retries.

Worked example: a claims project

1. Parent: *Child name* **Claims**, *Scope* **Around entities**, *Start* **APP-0002**, *Hops* **1** > **Create child**. Folder EA_Model_Sample_Claims appears next to EA_Model_Sample.
2. Close the parent; open EA_Model_Sample_Claims\EA_Entities.xlsm: "Child model Claims is ready" with Claims Platform, its five neighbours and their references.
3. In the child: Claims Platform *Vendor* → Duck Creek; add Fraud Engine serving Claims Department; Claim Record *Owner* → Claims Data. **Prepare merge package**, close.
4. Meanwhile, in the parent: Claims Platform *Version* → 9.1; Claim Record *Owner* → Data Governance.
5. Parent: **Choose child and preview**. MERGE_Review: Vendor Update (Child); Fraud Engine Add; the Serving relationship Add; Claim Record Owner Conflict (Parent: Data Governance, Child: Claims Data); no row for Version.
6. Set the conflict to **Child** and **Apply merge**. Claims Platform has Vendor Duck Creek and Version 9.1; Claim Record's Owner is Claims Data; Fraud Engine and its relationship are in the parent.
7. Preview again straight away: nothing to merge.

14. Reference

The tables below describe the metamodel shipped with 1.1 and every sheet in the two workbooks; your model may have more types, attributes and lists if they have been added.

Entity types by domain (Display ID prefix)

Domain	Entity types
Common	Role (ROL), Collaboration (COL), Path (PTH), Service (SRV), Process (PRC), Function (FUN), Event (EVT), Grouping (GRP), Location (LOC)
Business	Business Actor (BAC), Business Interface (BIF), Business Object (BOB), Product (PRD)

Domain	Entity types
Application	Application Component (APP), Application Interface (AIF), Data Object (DOB)
Technology	Node (NOD), Device (DEV), System Software (SSW), Technology Interface (TIF), Communication Network (CNW), Distribution Network (DNW), Equipment (EQP), Facility (FAC), Artifact (ART), Material (MAT)
Strategy	Resource (RES), Capability (CAP), Value Stream (VST), Course of Action (COA)
Motivation	Stakeholder (STK), Driver (DRV), Assessment (ASM), Goal (GOL), Outcome (OUT), Principle (PRI), Requirement (REQ), Meaning (MNG), Value (VAL)
Implementation & Migration	Work Package (WPK), Deliverable (DLV), Plateau (PLT)

ArchiMate 4 moves Role, Collaboration, Process, Function, Event, Service and Path into a Common domain shared by the layers; the *Domain* attribute records which layer such an element belongs to.

Attributes

Name	Code	Data type (list)	Meaning
Owner	Owner	Text	Accountable person or team
Lifecycle	Lifecycle	List (Lifecycle)	Lifecycle stage
Landscape Level	LandscapeLevel	List (LandscapeLevel)	TOGAF landscape level
Architecture State	ArchitectureState	List (ArchitectureState)	Baseline, transition or target
Domain	Domain	List (DomainList)	Layer of a Common domain element
Criticality	Criticality	Integer	Business criticality, 1 low to 5 high
Vendor	Vendor	Text	Supplier or vendor

Name	Code	Data type (list)	Meaning
Version	Version	Text	Version number, such as 4.2.1
Hosting	Hosting	List (Hosting)	Hosting model
Go-live Date	GoLiveDate	Date	Date put into production
Retirement Date	RetirementDate	Date	Planned or actual retirement
Annual Cost	AnnualCost	Currency	Annual run cost
Data Classification	DataClassification	List (DataClassification)	Information classification
Process Level	ProcessLevel	Integer	Process hierarchy level, 0 to 5
Maturity	Maturity	List (Maturity)	Capability maturity
Capability Level	CapabilityLevel	Integer	Capability map level, 1 to 4
Priority	Priority	List (Priority)	MoSCoW priority
Start Date	StartDate	Date	Start date
End Date	EndDate	Date	End date
Measure	Measure	Text	How achievement is measured
URL	Url	URL	Link to documentation

Which attributes apply to which type, and which are required, is on MM_EntityAttributes and on each type's form. For example, Application Component has Owner* and Lifecycle* (required), Landscape Level, Architecture State, Criticality, Vendor, Version, Hosting, Go-live Date, Retirement Date, Annual Cost and URL.

Lists

List	Values
Status	Active, Retired
Lifecycle	Plan, Phase In, Production, Phase Out, Retiring, Retired

List	Values
RepositoryArea	Architecture Metamodel, Architecture Capability, Architecture Landscape, Standards Information Base, Reference Library, Governance Log, Architecture Requirements Repository, Solutions Landscape
LandscapeLevel	Strategic, Segment, Capability
ArchitectureState	Baseline, Transition, Target
Hosting	On-premises, SaaS, IaaS, PaaS, Hybrid
DataClassification	Public, Internal, Confidential, Restricted
Priority	Must, Should, Could, Won't
Maturity	1 - Initial, 2 - Managed, 3 - Defined, 4 - Measured, 5 - Optimised
DomainList	Business, Application, Technology

Sheets in EA_Core

Sheet	Purpose
Home	Buttons and counts
MM_EntityTypes, MM_AttributeTypes, MM_RelationshipTypes, MM_EntityAttributes, MM_EntityRelationships, MM_TypeGroups, MM_Rules, MM_Lists, MM_Styles	The metamodel (section 7)
DATA_Master	One wide row per entity, all attributes as columns; not-applicable cells shaded
DATA_Relationships	One row per relationship
H_<type>, H_Relationships, H_Metamodel	Change history (section 9)
SYS_Issues	Validate All results (section 6)
SYNC_Review	Differences waiting for a decision (section 8)
SYS_Versions, VERSION_Compare	Snapshot register and compare results (section 9)

Sheet	Purpose
IMPORT_Preview	Import preview (section 10)
SYS_Views, HARVEST_Review	Views register and harvest review (section 12)
MERGE_Review, SYS_Children	Merge review and child register (section 13)
FORM_Entity, FORM_Relationship, FORM_Type, FORM_Rule, FORM_Versions, FORM_IO, FORM_Diagram, FORM_Merge	The forms, drawn by the buttons
SYS_Settings	Settings (Setup Guide)
SYS_Counters, SYS_ListCache, SYS_MapCache	Display ID counters and caches; do not edit

Sheets in EA_Entities

Sheet	Purpose
Home	Buttons, counts and the list of type sheets
T_<type>	One per entity type (section 3)
RPT_EntityDetail, RPT_Impact	Reports, created when first used (section 11)
SYS_Lists, SYS_RuleCache, SYS_Pending, SYS_SyncState	Dropdown lists, rule cache, queued edits and sync state; do not edit

15. Good practice and FAQ

A few habits keep a repository trustworthy; the questions below are the ones new users ask most.

Good practice

- **Name things the way the business does,** and keep names unique within a type: imports, diagrams and merges match on names when there is no ID.
- **Fill required attributes as you go.** A row that waits for an owner stays red and unsent.
- **Retire, never delete.** Retired entities keep the history readable and the diagrams honest.

- **Run Sync Now and Validate All weekly,** and before every review board, export or merge.
- **Take a labelled snapshot before big changes,** such as "Before Q3 portfolio review"; automatic snapshots cover the risky operations, but a label is easier to find.
- **Change the metamodel deliberately.** One person owns it; changes go through the forms, followed by Apply metamodel changes, a stencil rebuild and fresh templates.
- **Drag existing elements from the ModelLibrary** instead of drawing them again, and harvest diagrams soon after drawing them.
- **Keep each model in its own local folder,** inside a trusted location, and back the folder up (the History folder holds the snapshots).

FAQ

Can two people work on the same model at the same time? Not in the same files: Excel workbooks are single-user. Give each person or team a child model and merge it back, or take turns with the folder.

Can I use the model on a Mac or in Excel on the web? You can open and read the workbooks, and edit EA_Entities, but no macros run. The next time the model is opened on Windows with macros, the outside edits are listed on SYNC_Review.

Can I put the model on OneDrive or SharePoint? Keep the working copy in a local folder; the Workbench refuses to run from a web location because files are renamed, moved and opened side by side. You can copy the folder to OneDrive as a backup.

I deleted a row by mistake. Nothing is lost: the next sync puts it back. To remove an entity, set it to Retired.

A cell is red but the value looks right. Hover over it for the message. Common causes: a number stored as text, a date in a format Excel did not recognise, or a list value with a typing difference.

Why is my new row not getting a Display ID? It has an error. Fix the red cell and save again.

The relationship I want is not offered. The metamodel does not allow it for those types. Check the allowed relationships (section 5) and, if the rule is missing, add it on the relationship type form (section 7).

Can I add my own entity types and attributes? Yes, with Add / edit type and Apply metamodel changes (section 7). They get type sheets, form fields, templates, draw.io shapes and report columns automatically.

How do I get a picture of part of the architecture for a slide? Generate a diagram around the entity (section 12), tidy it in draw.io, then Export images.

EA_Reports shows old numbers. It reads the saved EA_Core. Click Refresh reports in the Workbench, or save EA_Core and use Data > Refresh All.

Where is everything kept? In the model folder: the two workbooks, History\ for snapshots and archives, Diagrams\ and Library\ for draw.io, Reports\ for Power BI, and the working folders Templates, Imports, Exports\ and Merge\ once used. Copying the whole folder copies the whole model.